

Trabajo Práctico de Procesamiento Avanzado de Imágenes

- Ecualización local de histograma
- Corrección de multiple choice

Integrantes:

- CASTILLO, Juan Ignacio
- FASOLATO, Matías
- TORREGIANI, Bautista

Profesor: Gonzalo Daniel Sad

ÍNDICE GENERAL

1. INTRODUCCIÓN	3
2. PROBLEMA 1 – ECUALIZACIÓN LOCAL DE HISTOGRAMA	4
2.1. Introducción de la imagen	4
2.2. Objetivo	5
2.3. ¿Por qué la ecualización global no es suficiente?	5
2.4. Implementación del algoritmo	6
2.5. Manejo de bordes	6
2.6. Cálculo del histograma y la CDF	7
2.7. Mapeo del píxel central	7
2.8. Influencia del tamaño de ventana	8
2.9. Costo computacional	9
2.10. Detalles encontrados	9
2.11. Conclusión del Problema 1	10
3. PROBLEMA 2 – CORRECCIÓN AUTOMÁTICA DE EXÁMENES MULTIPLE CHOICE	11
3.1. Objetivo	11
3.2. Binarización de la imagen	11
3.3. Detección automática de las celdas del examen	12
3.4. Localización de la respuesta dentro de cada pregunta	14
3.5. Filtrado de ruido	14
3.6. Reconocimiento de las letras A, B, C y D	14
3.7. Problema con la conectividad 8	15
3.8. Distinción entre A y D	15
3.9. Pruebas con respuestas vacías y múltiples	16
3.10. Corrección del examen	16
3.11. Validación del encabezado	16
3.12. Detección automática	16
3.13. Validación de Name	17
3.14. Validación de Date	18
3.15. Validación de Class	18
3.16. Procesamiento automático de todos los exámenes	18
3.17. Generación de la imagen resumen	19
4. CONCLUSIONES GENERALES	21

1- Introducción

En el campo del Procesamiento Digital de Imágenes, la ecualización de histograma es una de las técnicas más utilizadas para mejorar el contraste de una imagen, redistribuyendo sus niveles de intensidad de manera más uniforme. Sin embargo, cuando una imagen presenta detalles cuya intensidad es similar a la de su entorno inmediato, una ecualización global (que utiliza el histograma de la totalidad de la imagen) puede no ser suficiente para resaltarlos. Por otro lado, la detección automática de estructuras dentro de una imagen, como texto o marcas manuscritas, es un problema clásico de visión por computadora con aplicaciones directas en la automatización de tareas administrativas, como la corrección de exámenes.

En este trabajo práctico se resolvieron dos problemas mediante técnicas de procesamiento digital de imágenes implementadas en Python, utilizando las bibliotecas OpenCV, NumPy y Matplotlib.

El primer problema consistió en implementar manualmente una ecualización local de histograma sobre una imagen en escala de grises de 8 bits, con el objetivo de revelar detalles ocultos en distintas regiones cuya intensidad era similar a la de su fondo local. A diferencia de la ecualización global, la transformación se calculó utilizando únicamente el vecindario de cada píxel, lo que permitió conservar la información local de la imagen. Se estudió además cómo influye el tamaño de la ventana utilizada sobre el resultado obtenido.

El segundo problema consistió en desarrollar un sistema capaz de corregir automáticamente exámenes de tipo multiple choice a partir de una imagen, sin utilizar coordenadas predeterminadas para localizar las preguntas. Además de corregir las diez respuestas de cada examen, el sistema debía validar los campos Name, Date y Class, procesar la totalidad de los exámenes disponibles, y generar una imagen resumen que diferenciara a los alumnos aprobados de los desaprobados.

El presente informe se organiza en dos secciones principales: en primer lugar se desarrolla la resolución del Problema 1, y posteriormente la del Problema 2, presentando en cada caso la metodología implementada, los resultados obtenidos y las conclusiones correspondientes.

2- Problema 1 – Ecualización local de histograma

Introducción de la imagen

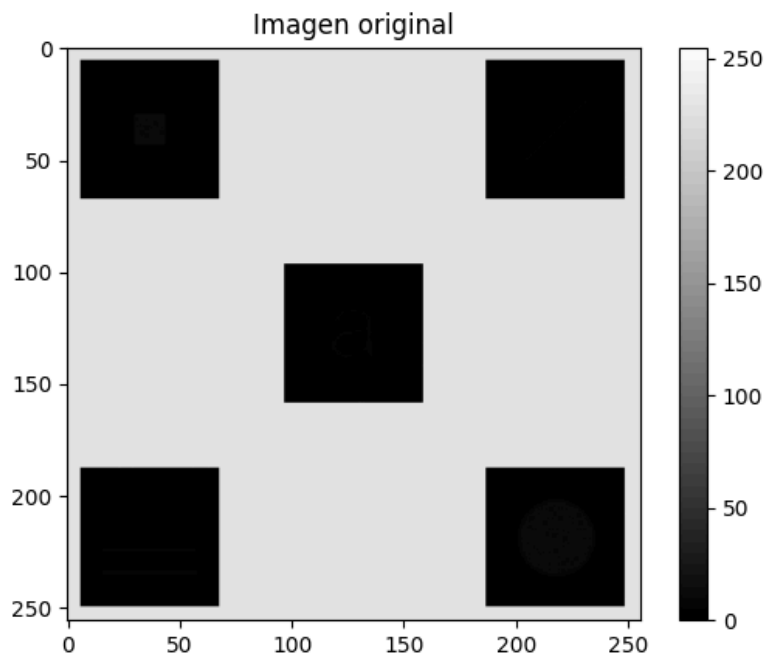


Figura 1. Imagen original utilizada para el análisis.

La imagen utilizada para el análisis es una imagen en escala de grises de 256×256 píxeles, lo que representa un total de 65.536 píxeles. Se encuentra almacenada utilizando el tipo de dato `uint8`, por lo que cada píxel puede representarse mediante valores enteros entre 0 y 255.

En este caso, los valores de intensidad presentes en la imagen se encuentran concentrados en dos regiones muy diferenciadas: se observan únicamente 15 niveles de gris, correspondientes a los valores 0 a 11 y 226 a 228. Por lo tanto, existe una fuerte concentración de píxeles en intensidades muy bajas y muy altas, sin presencia de valores intermedios. La intensidad mínima registrada es 0, mientras que la máxima es 228.

La intensidad media de la imagen es de aproximadamente 160,66, con un desvío estándar de 103,00. Este desvío elevado respecto de la media se relaciona con la marcada separación entre los niveles de intensidad bajos y altos presentes en la imagen.



Figura 2. Histograma global de la imagen original.

El histograma de la imagen original presenta una distribución fuertemente bimodal, donde la totalidad de la información se concentra en dos extremos del rango dinámico mientras los niveles intermedios de gris son nulos.

Objetivo

El objetivo fue implementar manualmente un algoritmo de ecualización local que recibiera como entrada una imagen y el tamaño de una ventana $M \times N$. Para cada píxel de la imagen se debía analizar únicamente el vecindario correspondiente a esa ventana, calcular su histograma local y utilizarlo para modificar el píxel central.

Las características mencionadas respecto a la imagen “Imagen_con_detalles_escondidos.tif” resultan particularmente relevantes para el objetivo del ejercicio, ya que la imagen contiene detalles escondidos en diferentes zonas, cuyos niveles de intensidad pueden ser similares a los del fondo local.

¿Por qué la ecualización global no es suficiente?

La ecualización global asigna una única transformación a toda la imagen, calculada a partir del histograma de la totalidad de los píxeles. El problema es que dentro de una región local pequeña el rango de intensidades puede ser muy distinto al de la imagen completa: un valor que a nivel global es muy bajo (por ejemplo, cercano al fondo oscuro) puede representar, dentro de su propio vecindario, uno de los valores más altos presentes en esa zona. La ecualización global no distingue esta diferencia relativa, porque sólo tiene en cuenta la posición del valor dentro del histograma completo de la imagen. La ecualización local, en cambio, recalcula el histograma en cada ventana, y por lo tanto sí es capaz de resaltar variaciones de intensidad que son relevantes localmente aunque sean insignificantes a escala global.

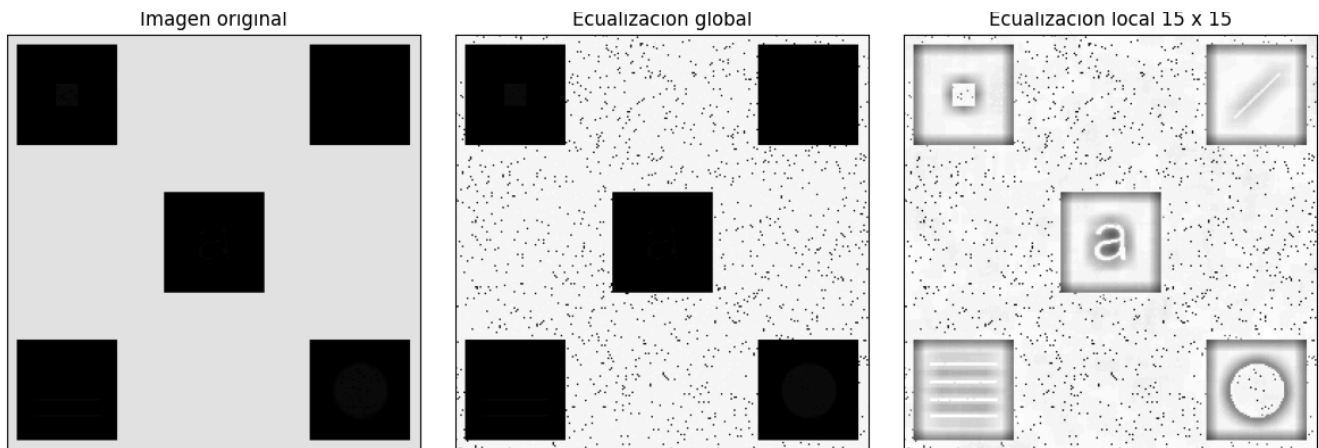


Figura 3. Comparación en 3 paneles: imagen original — ecualización global — ecualización local 15×15.

Implementación del algoritmo

La imagen fue cargada en escala de grises, de manera que cada píxel estuviera representado por un valor de intensidad entre 0 y 255, en formato `uint8`.

Manejo de bordes

Para poder centrar la ventana también sobre los píxeles cercanos a los bordes, se agregó un margen alrededor de la imagen mediante `cv2.copyMakeBorder()`, utilizando el modo `cv2.BORDER_REPLICATE`. Se eligió este modo en particular, y no otras alternativas como `BORDER_CONSTANT` o `BORDER_REFLECT`, porque:

- `BORDER_CONSTANT` introduciría un valor artificial (típicamente 0) en el margen, lo que alteraría el histograma local calculado cerca del borde con intensidades que no pertenecen realmente a la imagen.
- `BORDER_REFLECT` duplicaría el patrón de píxeles cercano al borde, lo que podría sesgar artificialmente la forma del histograma local en esa zona.
- `BORDER_REPLICATE`, en cambio, extiende el último valor real de píxel hacia afuera, por lo que el histograma local en los bordes sigue reflejando fielmente la intensidad real de esa región de la imagen.

Se trabajó con dimensiones impares para la ventana (7, 15, 31, 63), ya que esto permite disponer de un único píxel central sin ambigüedad. Por ejemplo, en una ventana de 31×31, existen 15 píxeles a cada lado del píxel central.

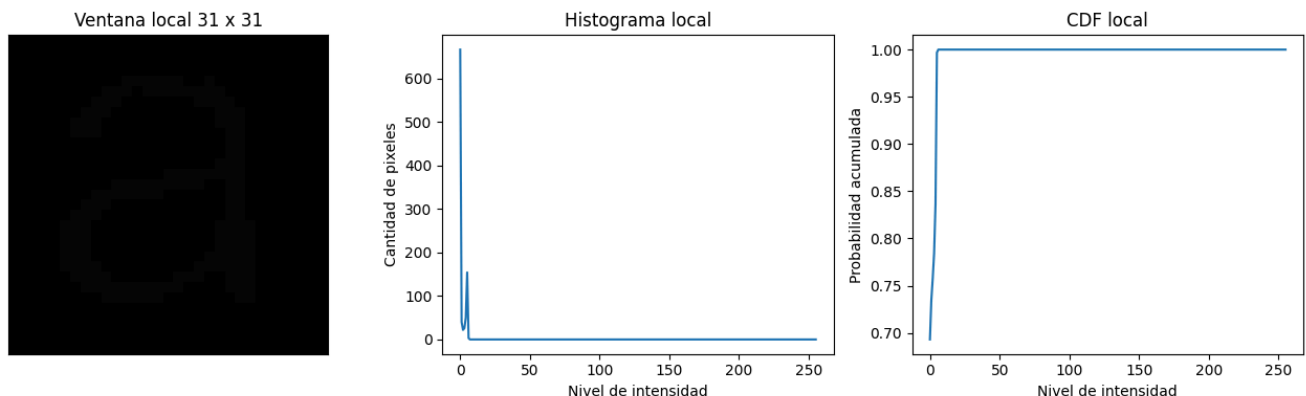
Cálculo del histograma y la CDF

Para cada posición de la imagen se extrajo el vecindario correspondiente y se calculó su histograma con 256 niveles de intensidad. El histograma se normalizó dividiendo la frecuencia de cada nivel por la cantidad total de píxeles de la ventana; para que esta división no se trunque a cero (numpy realiza división entera entre arrays de tipo entero), el histograma se convirtió previamente a tipo `double`. A partir del histograma normalizado se calculó la función de distribución acumulada (CDF) mediante la suma acumulada (`cumsum`), procedimiento equivalente al utilizado en el material teórico de la materia.

Mapeo del píxel central

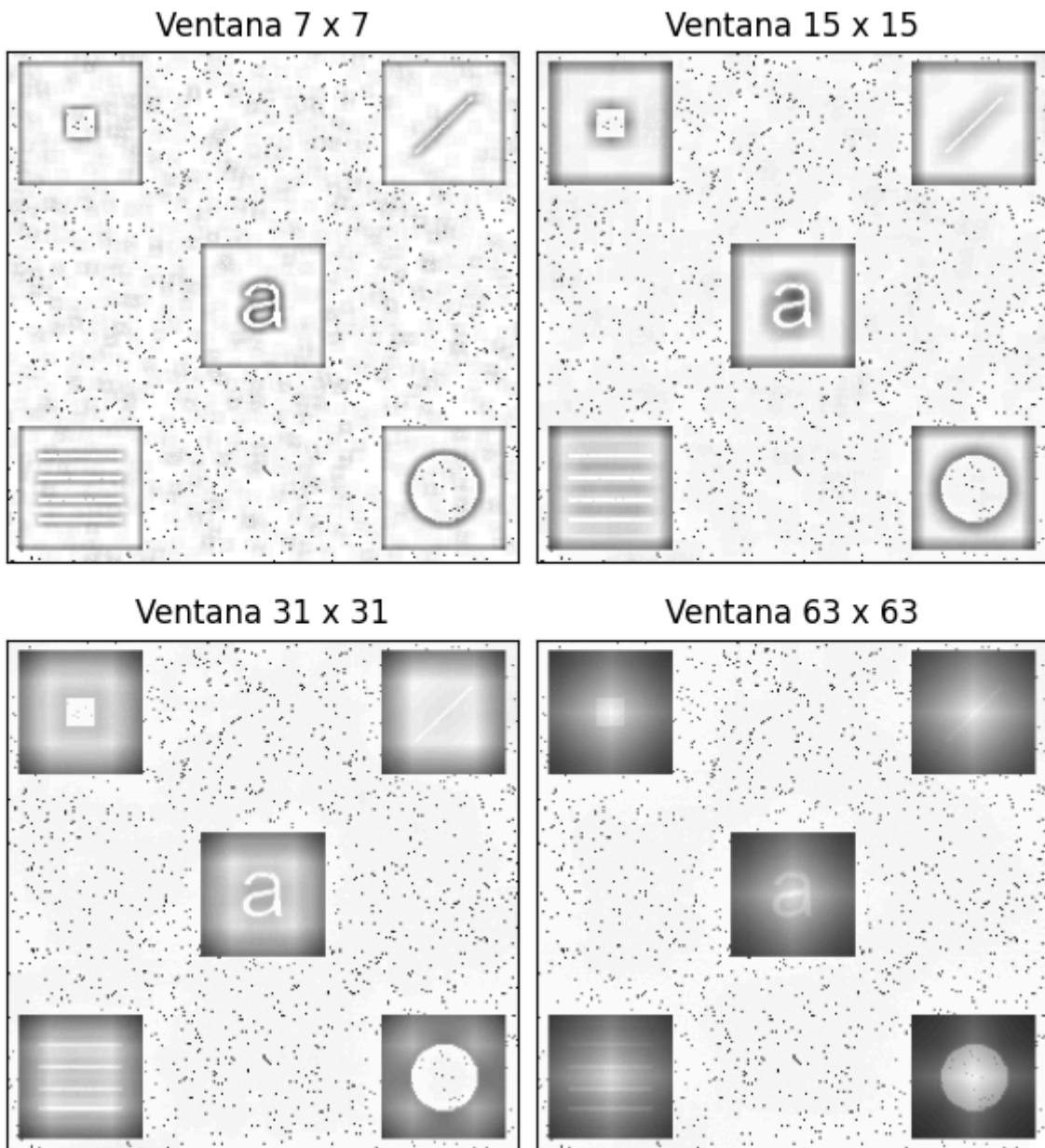
Un punto importante del algoritmo es que la ventana se extrae de la imagen *con relleno* (`img_pad`), pero el valor de intensidad que se transforma se toma de la imagen *original* en la posición (`fila`, `columna`). Esto es posible porque el margen agregado desplaza exactamente la ventana extraída de `img_pad` para que quede centrada en la posición correspondiente de la imagen original — es decir, la ventana "ve" el contexto con bordes replicados, pero el píxel que efectivamente se transforma es siempre uno real de la imagen. Para ese valor se buscó su posición correspondiente en la CDF local y se lo llevó nuevamente al rango de 0 a 255 (multiplicando por 255 y redondeando). Este resultado se almacenó en la posición correspondiente de la imagen de salida.

Este procedimiento se repitió píxel por píxel hasta recorrer toda la imagen ($256 \times 256 = 65.536$ iteraciones, cada una con el cálculo completo de un histograma de 256 niveles). Por lo tanto, a diferencia de una ecualización global, cada píxel fue transformado utilizando un histograma diferente, correspondiente a su entorno local. Esta implementación prioriza la claridad conceptual del algoritmo por sobre la eficiencia: al recalcular el histograma completo en cada posición sin reutilizar información entre píxeles vecinos, el costo computacional crece de forma proporcional al área de la ventana ($M \times N$), lo cual se discute en la sección siguiente.



Influencia del tamaño de ventana

Para estudiar la influencia del tamaño del vecindario se probaron distintas ventanas, particularmente tamaños de 7×7 , 15×15 , 31×31 y 63×63 .



Las ventanas pequeñas (7×7) producen una transformación extremadamente local: al contener solo 49 píxeles distribuidos potencialmente entre 256 niveles posibles de intensidad, el histograma resultante queda disperso, con la mayoría de los niveles vacíos o con una o dos cuentas. Esto genera una CDF muy irregular ("escalonada"), que amplifica diferencias de intensidad mínimas de forma desproporcionada. En la práctica esto se traduce en un realce muy marcado, pero también en ruido visible, ya que variaciones que no corresponden a detalles reales terminan siendo también amplificadas.

Al incrementar el tamaño de la ventana se incluye más información del entorno en el histograma, lo que produce una CDF más suave y estable. En las pruebas realizadas, la ventana de 15×15 ofreció el mejor equilibrio: los detalles escondidos se distinguen con claridad, se mantiene una buena definición de sus bordes y el ruido introducido por ventanas más chicas se reduce considerablemente.

Al utilizar ventanas considerablemente mayores (31×31 y, en particular, 63×63), el comportamiento empieza a aproximarse al de una ecualización menos local, ya que el histograma considera una región mucho más extensa y heterogénea. Por este motivo, algunos contrastes locales dejan de resaltarse con la misma intensidad y ciertos detalles pierden definición en sus bordes.

Costo computacional

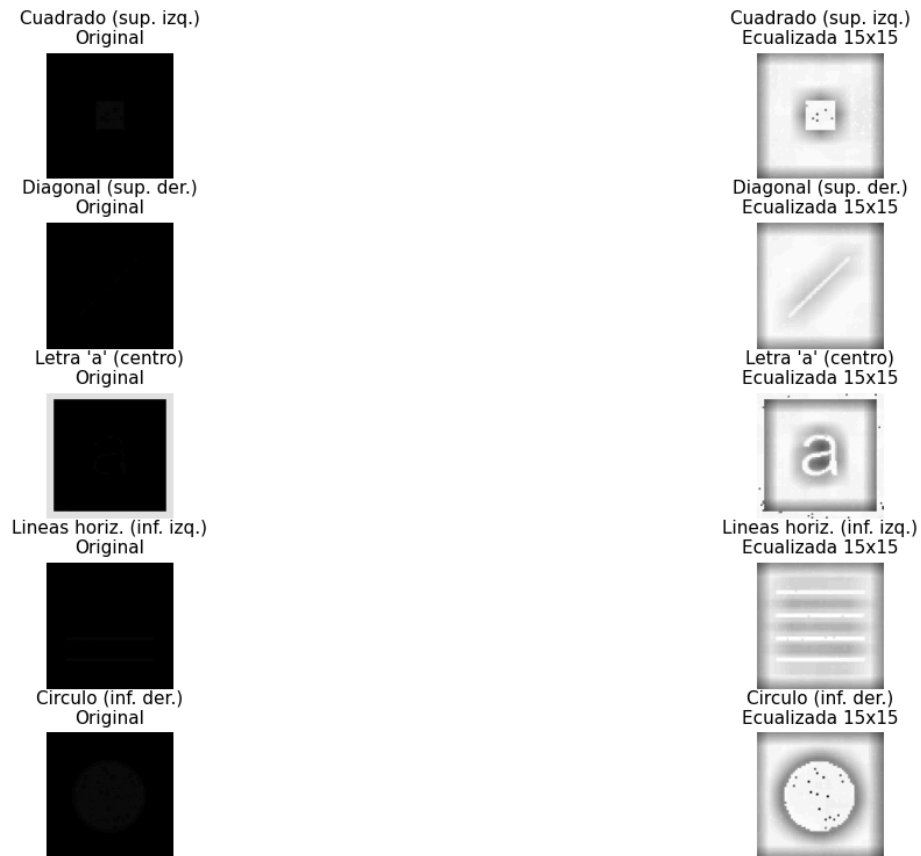
El algoritmo implementado recalcula un histograma de 256 niveles para cada uno de los 65.536 píxeles de la imagen, sobre una ventana cuyo tamaño crece de forma cuadrática con M . Esto implica que el tiempo de procesamiento aumenta notablemente al pasar de ventanas de 7×7 a 63×63 , ya que cada histograma individual debe recorrer una cantidad de píxeles significativamente mayor.

La comparación de diferentes ventanas fue importante porque permitió verificar experimentalmente que no existe un tamaño universalmente óptimo: el resultado depende tanto de la escala de los detalles que se desean destacar como del compromiso deseado entre realce, ruido y costo computacional.

Detalles encontrados

Luego de aplicar la ecualización local (con ventana 15×15) fue posible distinguir elementos que prácticamente no podían observarse en la imagen original:

- Un pequeño cuadrado en la región superior izquierda.
- Una estructura diagonal en la región superior derecha.
- Una letra similar a una "a" en la zona central.
- Varias líneas horizontales en la parte inferior izquierda.
- Una figura circular en la región inferior derecha.



Estos resultados muestran la ventaja de utilizar información local: los objetos poseían intensidades muy similares a las de su entorno y, por lo tanto, no se destacaban correctamente mediante una transformación basada en toda la imagen — tal como se evidenció también en la comparación con la ecualización global (Figura 3).

Conclusión del Problema 1

La ecualización local permitió mejorar el contraste de regiones que originalmente presentaban diferencias de intensidad muy pequeñas respecto del fondo, algo que la ecualización global no logra distinguir porque su transformación depende únicamente de la posición relativa de cada valor dentro del histograma completo de la imagen.

El tamaño de la ventana resultó ser un parámetro fundamental: ventanas demasiado pequeñas (7×7) generan un realce muy localizado pero incrementan considerablemente el ruido, debido a que el histograma local queda disperso entre pocos píxeles. Al aumentar el tamaño a 15×15 , los detalles escondidos se distinguen con mayor claridad y se mantiene una buena definición de sus bordes, obteniéndose el mejor equilibrio entre realce y ruido. Con ventanas de 31×31 y, especialmente, 63×63 , las regiones alrededor de los objetos presentan variaciones de intensidad más amplias, algunos detalles pierden definición y comienzan a notarse posibles artefactos cerca de los bordes de la imagen debido al relleno replicado. Por este motivo, para esta imagen se consideró que la ventana de 15×15 proporciona el resultado visual más adecuado, considerando además que su costo computacional es sensiblemente menor que el de ventanas más grandes.

3- Problema 2 – Corrección automática de exámenes multiple choice

Objetivo

El segundo problema tuvo como objetivo desarrollar un algoritmo que recibiera únicamente la imagen del examen, sin utilizar las coordenadas de las preguntas como información conocida. La consigna establecía además que una pregunta con más de una respuesta marcada o sin ninguna respuesta debía considerarse incorrecta.

Las respuestas correctas utilizadas fueron:

Pregunta	1	2	3	4	5	6	7	8	9	10
Respuesta	C	B	A	D	B	B	A	B	D	D

Además, el encabezado debía cumplir las condiciones establecidas para Name, Date y Class: el nombre debía contener al menos dos palabras y como máximo 25 caracteres, la fecha debía contener ocho caracteres formando una sola palabra y la clase debía contener un único carácter.

Binarización de la imagen

El primer paso consistió en convertir el examen a una representación binaria que facilitara la separación entre fondo y contenido.

En la implementación se utilizó el método de **Otsu** mediante `cv2.threshold`, permitiendo determinar automáticamente el umbral adecuado para separar los píxeles claros y oscuros.

Posteriormente se generó una matriz donde los píxeles oscuros correspondientes a texto y líneas tomaban valor 1, mientras que el fondo tomaba valor 0. Esta representación simplificó los análisis posteriores mediante sumas de píxeles.

Imagen original

Name: ESTEBANALVAREZ Date: 11/07/24 Class: 1

1 The Earth's system that involves all our air is called the _____. A geosphere B hydrosphere C atmosphere D biosphere	6 The gaseous layers of the atmosphere are held to Earth's surface by _____. A their weight B gravity C the sun D none of the above
2 The Earth's system that involves all our water is called the _____. A geosphere B hydrosphere C atmosphere D biosphere	7 78% of the Earth's atmosphere is made up of _____. A nitrogen B oxygen C carbon dioxide D water vapor
3 The Earth's system that involves all our rock is called the _____. A geosphere B hydrosphere C atmosphere D biosphere	8 They layer of the atmosphere we live in is called the _____. A stratosphere. B troposphere. C mesosphere. D exosphere.
4 The Earth's system that involves all living things is called _____. A geosphere B hydrosphere C atmosphere D biosphere	9 Most life in the ocean is found _____. A throughout all its waters. B deep down in the depths. C far from shore. D on the surface and closer to shore.
5 97% of Earth's water is found in _____. A lakes B the ocean C our underground aquifers D the clouds	10 A biomes location on Earth depends upon: _____. A climate B amount of rainfall C temperature D all of the above

Umbralado - Otsu T=163

Name: ESTEBANALVAREZ Date: 11/07/24 Class: 1

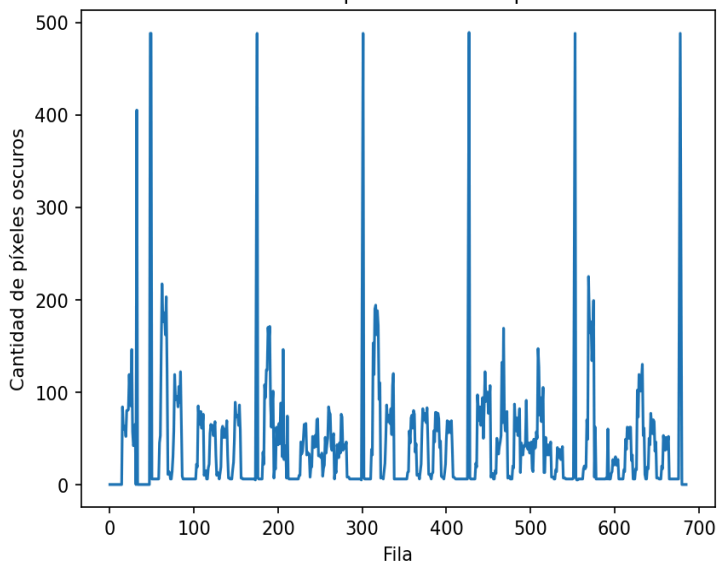
1 The Earth's system that involves all our air is called the _____. A geosphere B hydrosphere C atmosphere D biosphere	6 The gaseous layers of the atmosphere are held to Earth's surface by _____. A their weight B gravity C the sun D none of the above
2 The Earth's system that involves all our water is called the _____. A geosphere B hydrosphere C atmosphere D biosphere	7 78% of the Earth's atmosphere is made up of _____. A nitrogen B oxygen C carbon dioxide D water vapor
3 The Earth's system that involves all our rock is called the _____. A geosphere B hydrosphere C atmosphere D biosphere	8 They layer of the atmosphere we live in is called the _____. A stratosphere. B troposphere. C mesosphere. D exosphere.
4 The Earth's system that involves all living things is called _____. A geosphere B hydrosphere C atmosphere D biosphere	9 Most life in the ocean is found _____. A throughout all its waters. B deep down in the depths. C far from shore. D on the surface and closer to shore.
5 97% of Earth's water is found in _____. A lakes B the ocean C our underground aquifers D the clouds	10 A biomes location on Earth depends upon: _____. A climate B amount of rainfall C temperature D all of the above

Detección automática de las celdas del examen

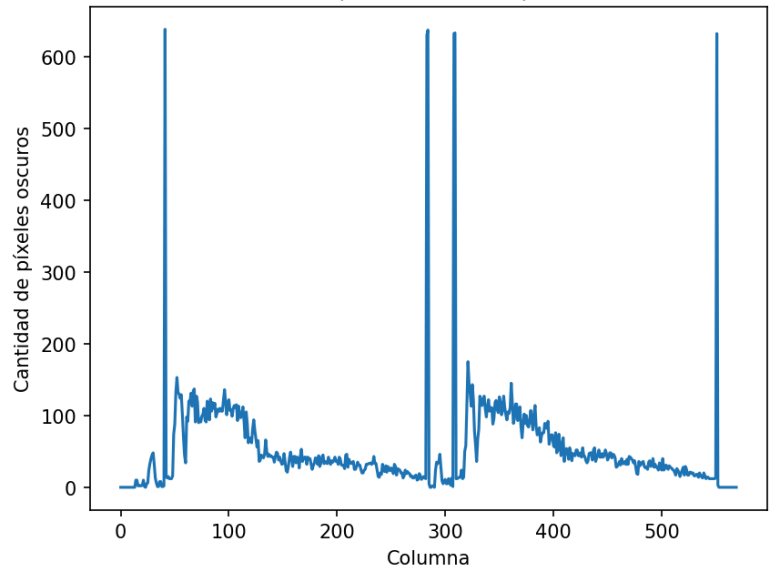
Para localizar la tabla sin utilizar coordenadas predeterminadas se sumaron los píxeles oscuros de cada fila y de cada columna.

Las líneas divisorias de la tabla contienen muchos más píxeles oscuros que las regiones con texto, por lo que producen picos claramente identificables en estas sumas. Esta estrategia coincide con la sugerida por la consigna para encontrar automáticamente las celdas.

Cantidad de píxeles oscuros por fila



Cantidad de píxeles oscuros por columna



Durante las primeras pruebas se utilizó un umbral equivalente al 70 % del máximo de la suma de filas. Sin embargo, este valor también detectaba como línea de la tabla parte del encabezado. Por este motivo se aumentó el umbral de las filas al 90 % del máximo, mientras que para las columnas se mantuvo en un 70 %.

Este fue uno de los primeros ajustes importantes del algoritmo: el umbral no se eligió arbitrariamente como resultado final, sino después de observar qué elementos se detectaban incorrectamente.

Como las líneas de la tabla podían tener más de un píxel de espesor, no alcanzaba con guardar todas las posiciones detectadas. Se utilizó `np.diff()` sobre el vector booleano de detección para identificar los cambios entre regiones sin línea y regiones con línea. Luego, mediante `np.argwhere()`, se obtuvieron los comienzos y finales de cada línea.

Con estos pares de coordenadas se recortaron automáticamente las diez preguntas: cinco de la columna izquierda y cinco de la columna derecha.

Pregunta 1

The Earth's system that involves all our air is called the _____.

A geosphere
B hydrosphere
C atmosphere
D biosphere

Pregunta 2

The Earth's system that involves all our water is called the _____.

A geosphere
B hydrosphere
C atmosphere
D biosphere

Pregunta 3

The Earth's system that involves all our rock is called the _____.

A geosphere
B hydrosphere
C atmosphere
D biosphere

Pregunta 4

The Earth's system that involves all living things is called _____.

A geosphere
B hydrosphere
C atmosphere
D biosphere

Pregunta 5

97% of Earth's water is found in _____.

A lakes
B the ocean
C our underground aquifers
D the clouds

Pregunta 6

The gaseous layers of the atmosphere are held to Earth's surface by _____.

A their weight
B gravity
C the sun
D none of the above

Pregunta 7

78% of the Earth's atmosphere is made up of _____.

A nitrogen
B oxygen
C carbon dioxide
D water vapor

Pregunta 8

The layer of the atmosphere we live in is called the _____.

A stratosphere.
B troposphere.
C mesosphere.
D exosphere.

Pregunta 9

Most life in the ocean is found _____.

A throughout all its waters.
B deep down in the depths.
C far from shore.
D on the surface and closer to shore.

Pregunta 10

A biomes location on Earth depends upon: _____.

A climate
B amount of rainfall
C temperature
D all of the above

Localización de la respuesta dentro de cada pregunta

Inicialmente fue necesario comprender correctamente dónde se encontraba la respuesta del alumno. Las letras correspondientes a las respuestas no estaban marcadas junto a las opciones A, B, C y D, sino escritas sobre la línea horizontal destinada a completar la respuesta dentro del enunciado.

Por este motivo, el algoritmo pasó a buscar automáticamente esa línea horizontal.

Dentro de cada celda se aplicó nuevamente una binarización y se obtuvieron componentes conectadas. Se consideraron candidatas a línea de respuesta aquellas componentes que fueran largas y muy finas. En particular, se utilizaron características como ancho mayor a 50 píxeles y altura menor o igual a 2 píxeles.

Una vez encontrada la línea, se eliminó únicamente esa componente utilizando la matriz de etiquetas devuelta por `connectedComponentsWithStats`. Luego se recortó una pequeña zona alrededor de ella, donde debía encontrarse la letra escrita por el alumno.

Filtrado de ruido

Dentro de la zona de respuesta se volvieron a obtener las componentes conectadas.

Durante las pruebas aparecieron componentes minúsculas producidas por ruido o pequeños artefactos de la imagen. Se analizaron las áreas de las componentes y se observó que las letras reales presentaban áreas considerablemente mayores. Por ejemplo, las respuestas válidas se encontraban aproximadamente entre 22 y 34 píxeles de área, mientras que algunos puntos de ruido tenían área igual a 1.

Se decidió conservar únicamente componentes con:

$$\text{área} > 10$$

Este mismo criterio de filtrado se utilizó posteriormente en algunas partes del encabezado. La implementación final conserva únicamente las componentes que superan ese umbral.

De esta manera se pudieron distinguir naturalmente los tres casos de una pregunta: cero componentes significa que no hubo respuesta, una componente representa una respuesta simple y dos o más componentes representan una respuesta múltiple.

Reconocimiento de las letras A, B, C y D

Una vez aisladas las letras surgió el problema de identificar cuál de las cuatro opciones representaba cada una.

En lugar de utilizar OCR, se buscaron características geométricas simples de las letras. La propiedad más útil fue la **cantidad de huecos cerrados**.

La lógica observada fue que la B contiene dos huecos, la C no contiene ninguno y tanto A como D contienen uno.

Problema con la conectividad 8

Durante la primera implementación se utilizó conectividad 8 para analizar el fondo de la letra. Esto provocó un error particularmente importante con la letra B.

La conectividad 8 considera vecinos no solamente los píxeles superior, inferior, izquierdo y derecho, sino también los píxeles ubicados diagonalmente. Como las letras aisladas eran muy pequeñas, aproximadamente de 7×10 píxeles, algunos píxeles del interior de los huecos podían conectarse diagonalmente con el fondo exterior.

Como consecuencia, un hueco que visualmente debía considerarse cerrado podía aparecer conectado con el exterior y el conteo era incorrecto.

Para resolverlo se hicieron dos cambios. Primero se agregó un borde negro de un píxel alrededor de la letra. Luego se cambió el análisis a conectividad 4, en la que solamente se consideran vecinos horizontales y verticales. La implementación final utiliza explícitamente `connectivity=4`.

Después se identificaron las componentes del fondo que tocaban el borde agregado. Estas correspondían al fondo exterior. Todas las componentes que no tocaban ese borde se consideraron huecos interiores de la letra.

Con esta modificación se obtuvieron correctamente dos huecos para B, cero para C y uno para A y D.

Distinción entre A y D

El número de huecos no permitía separar A de D, porque ambas poseen uno.

Se analizaron entonces las dimensiones de las componentes en distintos exámenes. En las imágenes disponibles, la A tenía un ancho aproximado de 9 píxeles, mientras que la D tenía un ancho aproximado de 7 píxeles.

A partir de estas pruebas se utilizó la condición:

$$\text{ancho} \geq 8 \rightarrow \mathbf{A}$$

y en caso contrario se clasificó como D. Esta regla se encuentra reflejada en la función final de reconocimiento.

Antes de adoptarla se verificó el comportamiento en varios exámenes, comprobando que las dimensiones se repetían.

Pruebas con respuestas vacías y múltiples

Para probar el algoritmo se utilizó especialmente el examen 2, ya que incluía distintos casos útiles: respuestas únicas, una pregunta con dos letras escritas y varias preguntas sin respuesta.

El algoritmo produjo correctamente cero componentes para las preguntas vacías y dos componentes para la pregunta con respuesta múltiple. Según la consigna, ambos casos se consideran directamente incorrectos.

Esto permitió comprobar que no era necesario reconocer qué letras aparecían cuando había varias respuestas: el solo hecho de encontrar más de una componente ya era suficiente para clasificar la pregunta como incorrecta.

Corrección del examen

Cada respuesta detectada se comparó con el vector de respuestas correctas.

Solo cuando existía exactamente una componente se ejecutaba el reconocimiento de letra. Si la letra coincidía con la respuesta esperada se incrementaba el contador de aciertos.

Finalmente se consideró aprobado al alumno que tuviera seis o más respuestas correctas, de acuerdo con la consigna.

Validación del encabezado

Detección automática

El encabezado se obtuvo utilizando la posición de la primera línea horizontal de la tabla. Como esta posición ya había sido detectada automáticamente, no fue necesario introducir coordenadas manualmente.

Se recortó toda la región comprendida entre la parte superior de la imagen y la primera línea de las preguntas. Luego, el encabezado se umbraló para obtener una imagen binaria, representando el texto y las líneas con valor 1 y el fondo con valor 0.

Dentro del encabezado se detectaron las tres líneas horizontales correspondientes a Name, Date y Class. Nuevamente se utilizaron componentes conectadas y se seleccionaron componentes largas y delgadas.

Las tres líneas se ordenaron según su posición horizontal. De izquierda a derecha correspondían a Name, Date y Class.

Posteriormente las líneas fueron eliminadas y se obtuvo un recorte independiente para cada campo.

Name	Date	Class
ESTEBANALVAREZ	11/07/24	1

Validación de Name

Para Name no era necesario reconocer qué letras habían sido escritas, sino determinar cuántos caracteres y cuántas palabras existían.

Se analizaron las componentes conectadas y se midieron las separaciones horizontales entre caracteres consecutivos.

Para determinar un umbral de separación entre palabras se realizaron pruebas con varios nombres válidos. En MARIA LOPEZ se observó una separación de aproximadamente 6 píxeles entre las dos palabras, mientras que en LUCAS FERNANDEZ y JUAN PEREZ la separación fue aproximadamente de 7 píxeles. Dentro de una misma palabra, las separaciones eran normalmente de apenas 1 o 2 píxeles.

A partir de estos resultados se consideró que una separación superior a 3 píxeles representa un espacio entre palabras. La función final utiliza precisamente esa condición.

De esta manera se pudo determinar el número de palabras sin utilizar reconocimiento óptico de caracteres.

Esto también permitió detectar correctamente casos inválidos. Por ejemplo, en un examen el nombre y apellido aparecían visualmente juntos, sin una separación suficiente, por lo que el sistema lo interpretó como una única palabra y clasificó el campo como MAL. Otro examen contenía únicamente MARIA, por lo que también fue considerado inválido.

Además se controló que la cantidad total de caracteres, incluyendo los espacios detectados, no superara 25.

Validación de Date

Para Date se requerían exactamente ocho caracteres formando una única palabra.

Uno de los casos más útiles para ajustar esta etapa fue el examen 5. La fecha estaba ligeramente desplazada verticalmente y apareció una pequeña componente adicional de ruido, de área aproximadamente igual a 5.

Se reutilizó el filtro **área > 10**, lo cual eliminó esta componente y dejó correctamente ocho caracteres válidos.

Luego se analizaron las distancias entre caracteres de fechas válidas en varios exámenes. Se observó que podían existir separaciones de hasta 5 píxeles sin que esto representara un espacio real. Por lo tanto, para Date se utilizó un umbral diferente del utilizado para Name: si la separación entre dos componentes supera 5 píxeles, el campo se interpreta como más de una palabra y se clasifica como incorrecto.

Esta diferencia entre los umbrales de Name y Date surgió directamente de las pruebas realizadas sobre las imágenes y evitó reutilizar una regla que no funcionaba correctamente para ambos campos.

El examen 4, donde la fecha estaba vacía, fue detectado correctamente como Date: MAL.

Validación de Class

La validación de Class fue la más simple.

Después de filtrar pequeñas componentes con el mismo criterio de área, el campo debía contener exactamente una componente válida. Si se encontraba una sola, el resultado era OK; cualquier otra cantidad producía MAL.

En los exámenes disponibles, este campo fue detectado correctamente en todos los casos.

Procesamiento automático de todos los exámenes

Una vez comprobado que las distintas partes funcionaban individualmente, se reorganizó el código en una función general `procesar_examen(ruta)`.

Esta función recibe únicamente la ruta de una imagen, detecta automáticamente la tabla, recorta las diez preguntas, corrige las respuestas, calcula la cantidad de aciertos, determina si el alumno aprobó, obtiene el encabezado, valida sus tres campos y devuelve además el recorte del nombre.

Posteriormente se ejecutó esta función en un ciclo para los cinco archivos examen_1.png a examen_5.png.

Los resultados obtenidos fueron:

Examen	Aciertos	Resultado	Name	Date	Class
1	0	DESAPROBADO	MAL	OK	OK
2	4	DESAPROBADO	MAL	OK	OK
3	10	APROBADO	OK	OK	OK
4	0	DESAPROBADO	OK	MAL	OK
5	10	APROBADO	OK	OK	OK

Por lo tanto, los exámenes 3 y 5 resultaron aprobados, mientras que los exámenes 1, 2 y 4 resultaron desaprobados.

Generación de la imagen resumen

Finalmente se generó una única imagen que contiene los recortes correspondientes al campo *Name* de los cinco exámenes y, junto a cada uno, el resultado obtenido.

El recorte del nombre se mostró en color verde si el examen estaba aprobado y en color rojo si estaba desaprobado, permitiendo diferenciar visualmente ambos grupos y cumpliendo el requisito de utilizar los crops originales de los nombres. Además, en la parte superior de la imagen se incluyó una referencia general ("Verde: Aprobado | Rojo: No aprobado") que indica explícitamente el significado de cada color. La consigna exige precisamente una salida que incluya estos recortes y permita distinguir los exámenes aprobados de los desaprobados.

En esta etapa se presentó otro inconveniente. Los recortes binarios de los nombres utilizaban únicamente los valores 0 (fondo) y 1 (tinta). Al mostrarlos directamente con `imshow`, Matplotlib les aplicaba su mapa de colores por defecto, con lo cual no era posible controlar qué color tomaba cada letra ni distinguir aprobados de desaprobados.

La solución fue construir, a partir del recorte binario, una imagen RGB propia: se generó una matriz de unos (fondo blanco) del mismo tamaño que el recorte, y luego se reemplazaron únicamente los píxeles correspondientes a la tinta (valor 1) por el color correspondiente a la condición del alumno (verde para aprobado, rojo para desaprobado).

De esta forma, cada recorte del nombre se visualizó con el fondo en blanco y las letras coloreadas según el resultado del examen, en lugar de depender del mapa de colores por defecto de Matplotlib.

Verde: Aprobado | Rojo: No aprobado

APROBADO

MARIA LOPEZ

APROBADO

JUAN PEREZ

DESAPROBADO

ESTEBAN ALVAREZ

DESAPROBADO

MARIA

DESAPROBADO

LUCAS FERNANDEZ

4- Conclusiones generales

El trabajo permitió aplicar distintos conceptos de procesamiento de imágenes de manera combinada.

En el primer problema se comprobó cómo una transformación basada en información local puede revelar estructuras que una operación global no permite distinguir con claridad. También se verificó experimentalmente que el tamaño del vecindario modifica significativamente el resultado y debe elegirse de acuerdo con la escala de los detalles de interés.

En el segundo problema se desarrolló un sistema completo de análisis de formularios sin utilizar coordenadas fijas de las preguntas. La solución se basó principalmente en binarización, proyecciones mediante sumas por filas y columnas, detección de transiciones, recortes automáticos y análisis de componentes conectadas.

Un aspecto importante del desarrollo fue que varios parámetros finales surgieron de *pruebas realizadas sobre distintos exámenes*. El umbral para detectar espacios entre palabras, la diferencia de ancho entre las letras A y D, el filtrado por área y la separación máxima admitida en Date no se establecieron únicamente de forma teórica, sino observando el comportamiento real de las componentes en varias imágenes.

También fue relevante el error encontrado al contar los huecos de las letras. El uso inicial de conectividad 8 permitía conexiones diagonales entre el interior de una letra y el fondo exterior, produciendo resultados incorrectos. El cambio a conectividad 4, junto con la incorporación de un borde alrededor de la letra, permitió resolver el problema y constituye un ejemplo de cómo la elección del criterio de conectividad puede modificar significativamente un análisis de componentes.

En conjunto, ambos problemas muestran que una solución de procesamiento de imágenes no consiste únicamente en aplicar una operación aislada, sino en construir una secuencia de etapas, observar los resultados intermedios, detectar casos problemáticos y ajustar los criterios hasta obtener un algoritmo capaz de comportarse correctamente frente a diferentes imágenes.